



Late Delivery Prediction

CS611 - Machine Learning Engineering
Group 10

Daarmi Sai Lanka
Huang Kai-Lin
Maanoj Shara
Ng Bao Ling Pauline
Shreya Venkataramana
Tan Yin Yun
Zhuang Baohong

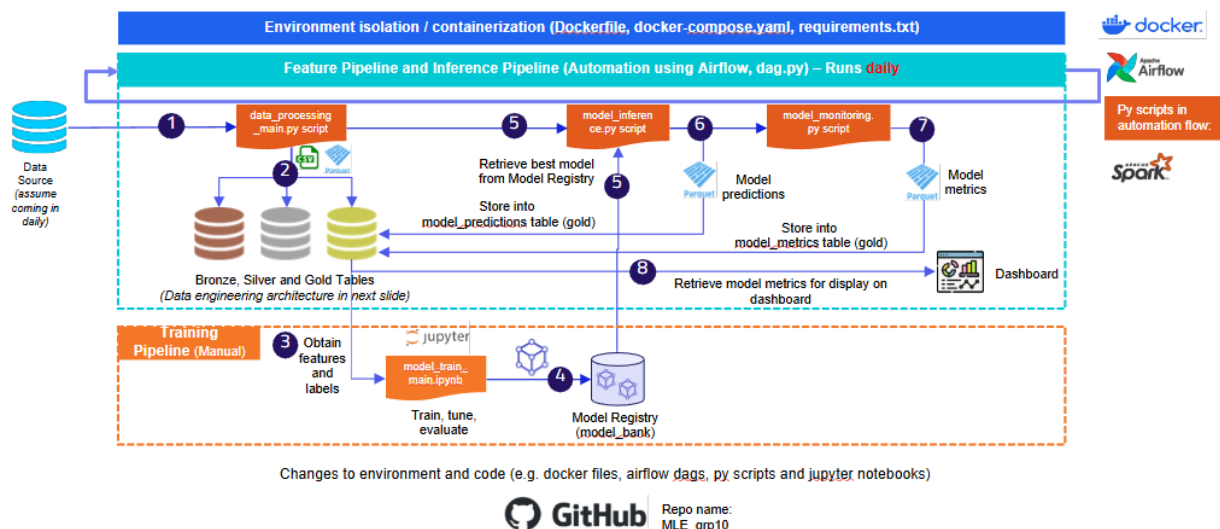
1. Introduction and Objective

Olist is an e-commerce SaaS platform, connecting small and medium enterprises (SMEs) in Brazil to various e-commerce platforms. As a unifying platform, SMEs are able to track their delivery performance at one glance. From exploratory analysis of the provided data (2016-2018), it was found that 27% of orders were delivered late (i.e. >14 days¹), resulting in an estimated R\$ 532,612 paid out in compensation. To improve customer satisfaction and reduce unnecessary losses, thus providing value-add via their SaaS platform, it is important for Olist to develop a feature that helps their SMEs predict whether an order will be late.

This project aims to address the business problem by developing a machine learning pipeline that (i) automates daily data processing, (ii) conducts predictive modelling and (iii) monitors model performance. Based on interviews with Olist clients, this solution should reduce the percentage of late orders from 27% to 20%, thus saving them R\$138,085 in paid compensation, a reasonable target.

2. Machine Learning (ML) Pipeline Design

As a SaaS provider whose solution may be installed in different OSes, it is important that the solution is portable. At the same time, given that e-commerce order data is often in high volume and velocity, it is crucial that the pipeline can handle big data. As such, the proposed architecture is containerized in Docker, using Pyspark for distributed data processing, and automated using Airflow daily to generate the required predictions for the Olist clients. Code management is done via Github.



The steps of the ML pipeline are as follows:

- Step 1, 2: Ingest and process raw data into datamart using medallion architecture (daily)
- Step 3: Train, tune and evaluate models
- Step 4: Save models into model registry
- Step 5: Retrieve best model from registry and data from datamart to make inferences
- Step 6: Store model predictions
- Step 7: Compute model monitoring metrics and store
- Step 8: Retrieve model metrics to display on dashboard

GitHub Repository: [MLE_grp10](#)

Docker environment configurations: [Dockerfile](#), [docker-compose.yaml](#), [requirements.txt](#)

¹ Average delivery time in Brazil is 16 days ([Statista, 2020](#))

3. Dataset

The dataset provided by Olist consists of 100,000 order data from 2016 to 2018. It comprises multiple tables, each of a different entity: customers, geolocation, order_items, order_payments, order_reviews, orders, products, sellers, product_category_name_translation. The csv files were distributed via [Kaggle](#).

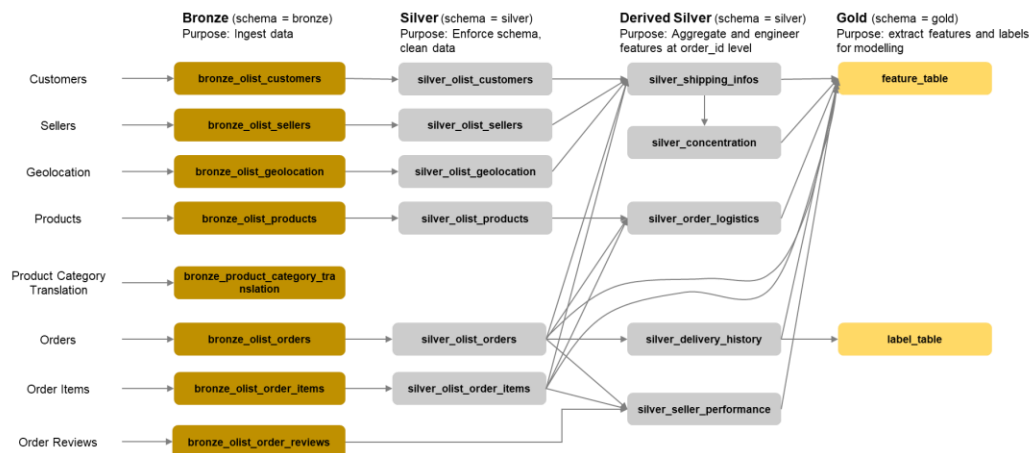
4. Data Processing

4.1 Logical Architecture

Data processing follows the medallion architecture. The approach taken is as follows:

- Bronze: Ingest raw data as-is for data lineage and audit purposes
- Silver: Enforce schema, clean data
- Derived Silver: Aggregate and engineer features, at order_id level, partitioned by date
- Gold: Extract relevant features and labels for modelling

The logical data architecture is shown below:



4.2 Table Schemas

Bronze Tables

A Pyspark script ([data_processing_bronze_table.py](#)) was written to ingest the raw data into parquet, with the exception of bronzeolist_orders, which was partitioned by order purchase date and each partition outputted into a csv file. All other table sizes were small, which meant partitioning was not necessary. (Note: Schema for the tables are same as silver tables below; just with “bronze_” instead of “silver_”)

Silver Tables

A Pyspark script ([data_processing_silver_table.py](#)) was written to clean the bronze tables and enforce the schemas. Only silverolist_orders table (follow its bronze counterpart) needed to be partitioned by date.

General principles adopted

- Missing values (e.g. in product_category_name) were replaced with “NaN” while numeric blanks were real NaN floats. This preserved analytic correctness without defaulting to zero
- A two-level classification – main category and sub category were manually created for categories
- All tables referencing user ids (e.g. customer_id, seller_ids) were verified to prevent downstream errors in aggregations and ensured every order can link to geographic, demographic attributes.
- Enums 'order_status' are standardized to maintain consistent categories over time and prevents unnoticed errors from affecting models

- Snapshot_date column is always added so queries can time-travel or partition-prune efficiently.

Table name	Field Name	Data Type	Data Processing Tasks
silverolist_customers	customer_id	string	Check for duplicates, write print statement
	customer_unique_id	string	
	customer_zip_code_prefix	string	Add missing leading zero in zipcodes
	customer_city	string	
	customer_state	string	
silverolist_sellers	seller_id	string	Check for duplicates, write print statement
	seller_zip_code_prefix	string	Add missing leading zero in zipcodes
	seller_city	string	
	seller_state	string	
silverolist_geolocation	geolocation_zip_code_prefix	string	Add missing leading zero in zipcodes
	geolocation_lat	float	Deduplicate per zipcode prefix by taking mean latitude, mean longitude
	geolocation_lng	float	
silverolist_products	product_id	string	Check for duplicates, write print statement
	product_category_name_portuguese	string	Input missing values as NaN
	product_category_name_english	string	Input missing values as NaN
	main_category	string	
	sub_category	string	
	product_name_length	double	Input missing values as NaN
	product_description_length	double	Input missing values as NaN
	product_photos_qty	double	Input missing values as NaN
	product_weight_g	double	
	product_length_cm	double	
	product_height_cm	double	
	product_width_cm	double	
silverolist_orders Partition by date (yyyy-mm-dd)	order_id	string	Remove invalid orders (those with no order items)
	customer_id	string	Check customer_id is valid against silverolist_customers
	order_status	string	Enforce enum (8 different statuses)
	order_purchase_timestamp	timestamp	
	order_approved_at	timestamp	
	order_delivered_carrier_date	timestamp	
	order_delivered_customer_date	timestamp	
	order_estimated_delivery_date	timestamp	
	snapshot_date	timestamp	
silverolist_order_items	order_id	string	
	order_item_id	long	

	product_id	string	
	seller_id	string	Check seller_id is valid against silver_olist_sellers
	shipping_limit_date	timestamp	
	price	double	
	freight_value	double	

Derived Silver Tables

A Pyspark script ([data_processing_silver_table.py](#)) was written to engineer features from the silver tables, as well as compute some aggregate metrics at the order_id level, partitioned by day. No schema was enforced at this stage as it is assumed the data types have already been enforced in the silver tables.

General principles adopted

- Derived tables are to house features that are computed from the silver transactional / event-based tables. For example, silver_delivery_history computes all the durations from the silver_olist_orders
- Order-level metrics (e.g total quantity, price, freight cost) are aggregated with reference to the silver_olist_orders table (the ground truth) on which orders to be included per partition, per order_id. This applies to silver_shipping_infos, silver_order_logistics
- To avoid data leakage, snapshot tables (e.g. silver_seller_performance and silver_concentration) only reference data before the partition date using a window function.

Table name	Field Name	Data Type	Data Processing Tasks
silver_shipping_infos Partition by date (yyyy-mm-dd)	order_id	string	
	order_purchase_timestamp	string	
	customer_zip_code_prefix	string	Extract from silver_olist_customers
	customer_city	string	Extract from silver_olist_customers
	customer_state	string	Extract from silver_olist_customers
	customer_lat	float	Extract from silver_olist_geolocation
	customer_lng	float	Extract from silver_olist_geolocation
	seller_zip_code_prefix	string	Extract from silver_olist_sellers
	seller_city	string	Extract from silver_olist_sellers
	seller_state	string	Extract from silver_olist_sellers
	seller_lat	float	Extract from geolocations
	seller_lng	float	Extract from geolocations
	delivery_distance	float	Compute Haversine distance (km)
	same_zipcode	boolean	customer_zip_code_prefix == seller_zip_code_prefix
	same_city	boolean	customer_city == seller_city
	same_state	boolean	customer_state == seller_state
	snapshot_date	date	
silver_order_logistics Partition by date (yyyy-mm-dd)	order_id	string	
	order_purchase_timestamp	timestamp	
	total_qty	integer	MAX(order_item_id) GROUP BY order_id
	total_price	float	SUM(price) GROUP BY order_id
	total_freight_value	float	SUM(freight_value) GROUP BY order_id

	total_weight_g	float	SUM(product_weight_g) by JOINING order_items.product_id = products.product_id GROUP BY order_id
	total_volume_cm3	float	SUM(product_length_cm * product_height_cm * product_width_cm) by JOINING order_items.product_id = products.product_id GROUP BY order_id
	total_density	float	total_weight_g / total_volume_cm3
	main_category	string	
	sub_category	string	
	snapshot_date	date	
silver_delivery_history Partition by date (yyyy-mm-dd)	order_id	string	
	order_purchase_timestamp	timestamp	
	is_weekend	integer	0 - weekday; 1 - weekend
	approval_duration	float	order_approved_at - order_purchase_timestamp
	processing_duration	float	order_delivered_carrier_date - order_approved_at
	ship_duration	float	order_delivered_customer_date - order_delivered_carrier_date
	act_days_to_deliver	integer	order_delivered_customer_date - order_purchase_timestamp
	miss_delivery_sla	boolean	act_days_to_deliver > 14 days
silver_seller_performance Partition by date (yyyy-mm-dd)	seller_id	string	
	avg_rating	float	Window function, average ratings (from bronze_olist_order_reviews) from orders before the partitioned date
	avg_delay_rate	float	Window function, average of miss_delivery_sla (from silver_delivery_history) before the partitioned date
	avg_processing_time	float	Window function, average of (order_delivered_carrier_date - order_purchase_timestamp) before the partitioned date
silver_concentration Partition by date (yyyy-mm-dd)	snapshot_date	timestamp	Extract month-year from order_purchase_timestamp
	granularity_level	string	Enum: city, state
	type	string	Enum: customer, seller
	concentration	float	For the snapshot_date, GROUP BY (granularity_level) then do WINDOW function on SUM(all customers) to compute the percentage of Total (i.e. the concentration) Same procedure is repeated for seller-level concentration

Gold Tables

A Pyspark script ([data_processing_gold_feature.py](#), [data_processing_gold_label_store.py](#)) was developed to engineer and aggregate business-ready features from the silver tables, producing gold tables optimized for analytics, reporting, and machine learning consumption.

General principles adopted

- Gold tables are designed to house features and aggregate metrics that are ready for direct consumption by ML models. For example, `gold_delivery_history` stores pre-computed delivery durations and summary statistics derived from `silver_olist_orders` and related silver tables.
- Order-level metrics such as total quantity, total price, and freight cost are aggregated and denormalized at the `order_id` and partitioned by day, referencing the cleaned and validated records in the `silver_olist_orders` table. This ensures that each gold table record represents a complete, business-ready snapshot for each order per partition date.
- To prevent data leakage, all snapshot and historical features are computed using only data available up to the partition date, leveraging window functions and as-of joins. This guarantees that no future data is included in the gold features, aligning with best practices for ML pipeline integrity.

Table name	Field Name	Data Type	Data Source
gold_feature_store	order_id	string	unique ID
	delivery_distance	float	silver_shipping_infos
	region_concentration	float	silver_concentration
	act_days_to_deliver	interger	silver_delivery_history
	same_city	boolean	silver_shipping_infos
	same_state	boolean	silver_shipping_infos
	day_of_week	integer	Sunday: 1, and so on
	is_weekend	boolean	silver_delivery_history
	season	string	Jul – Sep: Spring, and so on
	total_price	float	silver_order_logistics
	total_weight_g	float	silver_order_logistics
	total_volume_cm3	float	silver_order_logistics
	total_density	float	silver_order_logistics
	total_qty	ininteger	silver_order_logistics
	total_freight_value	float	silver_order_logistics
	avg_seller_rating	float	silver_seller_performance
	avg_delay_rate	float	silver_seller_performance
	avg_processing_time	float	silver_seller_performance
	seller_city	string	silver_shipping_infos
	seller_state	string	silver_shipping_infos
gold_label_store	order_id	string	unique ID
	label	Integer	If delivery_date – purchased_date > 14, label = 0; else, label = 1

5. Model Training

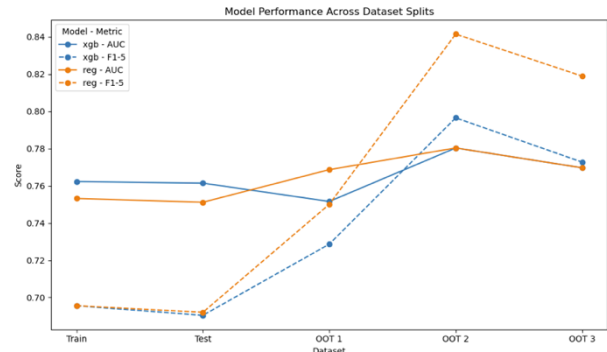
We selected Linear Regression and XGBoost for training due to their complementary strengths—Linear Regression offers simplicity and interpretability for linear patterns, while XGBoost excels in handling complex, non-linear data with high accuracy.



We evaluated both Linear Regression and XGBoost models across Train, Test, and Out-of-Time (OOT) datasets to assess their performance consistency and generalization capability. Our training strategy involved using 12 months of data to capture seasonality, while OOT1, OOT2, and OOT3 datasets were used to evaluate model robustness on future unseen periods. F1.5 score was selected as a key evaluation metric to place greater emphasis on recall, prioritizing the accurate identification of potential delivery failures.

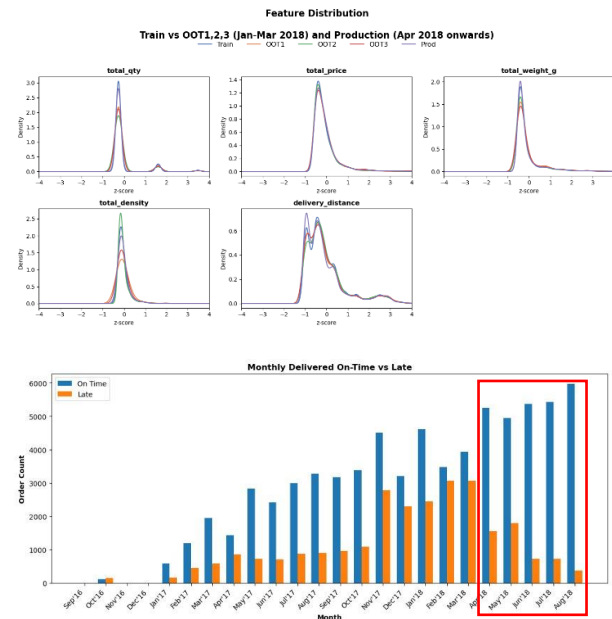
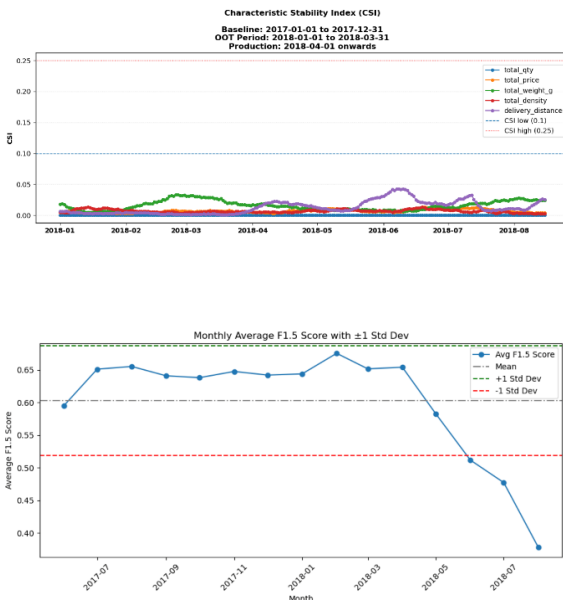
As shown in the performance chart, both models perform similarly across all dataset splits. However, Linear Regression demonstrates strong overall performance, particularly in F1.5 scores across the OOT datasets. While XGBoost shows slightly higher AUC in later OOT periods, the gains are marginal. Both models show improved performance from the Test to OOT periods, indicating enhanced generalization over time.

Given its competitive results, simplicity, and interpretability, Linear Regression is a strong candidate for deployment. XGBoost may still be considered when marginal gains in accuracy are required, especially in AUC-focused tasks.



6. Monitoring

During the Out-of-Time (OOT) and Production periods, we observed no significant data drift across key numeric features. Feature distributions between Train, OOT1–3, and Production remain consistent, as supported by the low Characteristic Stability Index (CSI) values, all remaining below the 0.1 threshold. This indicates strong input data stability, which is critical for maintaining model reliability post-deployment.

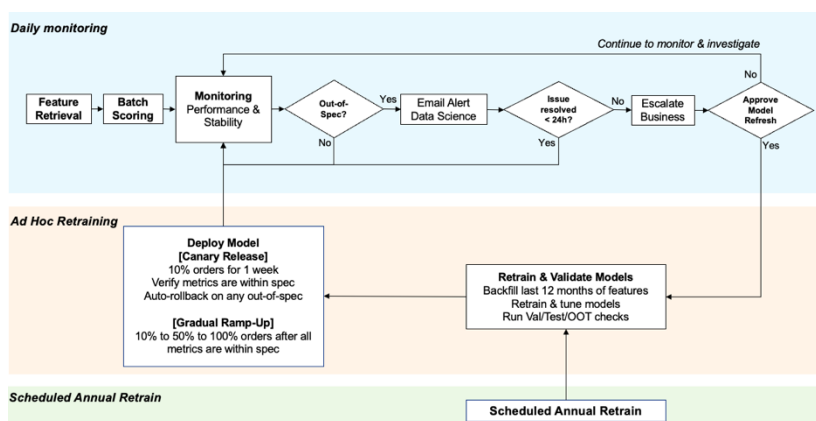


However, despite stable inputs, model performance—measured by monthly average F1.5 score—began to decline after Production launch in April 2018. By June 2018, F1.5 dropped below the defined threshold of 0.525, triggering the need for model retraining. This highlights the importance of continuous performance monitoring, as factors beyond feature distribution (e.g., changes in delivery operations or external conditions) may impact predictive effectiveness over time.

From our initial investigation, we observed a significant decrease in the percentage of late deliveries starting from April 2018. This shift likely contributed to the drop in model accuracy, as fewer positive cases impacted recall. To address this, additional data and supporting evidence from the business are needed to facilitate effective model retraining for the new operational environment.

7. Operational Monitoring and Deployment Strategy

Our pipeline retrieves features and performs batch scoring daily at 9am, then assesses performance and stability against predefined thresholds. Any out-of-specification metric triggers an immediate email alert to the data science team for further investigation. If the issue remains unresolved after 24 hours, it is escalated to business stakeholders for discussion and approval to retrain the model. This approach ensures detection of data drift and model degradation.

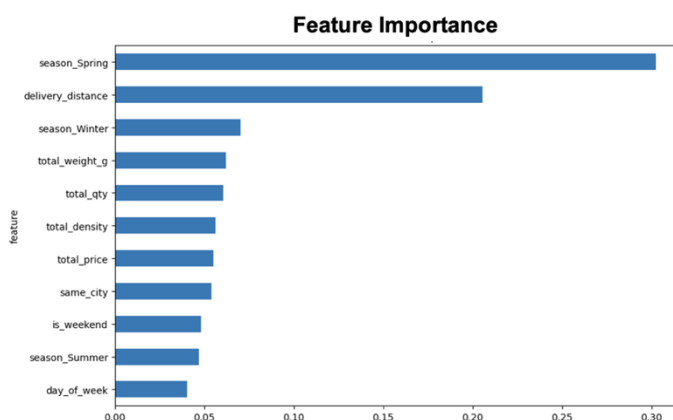


For model updates, we use a two-tiered retraining strategy. In ad hoc retraining, new models roll out 10% of traffic for one week via canary release, with automatic rollback on any threshold breach. After validation, traffic is ramped up incrementally to 100% when all metrics remain within specifications, minimizing user impact. Additionally, annual retrain is scheduled if the model has not been refreshed in the previous 12 months. Both ad hoc and scheduled annual retrain workflow backfill the latest 12 months of data, retrain and tune models and conduct rigorous validation, test and OOT checks before deployment.

8. Insights and Conclusion

As seen from the feature importance chart, seasonality is the most important feature, taking up 2 of 3 top spots and 'season_Spring' alone having a top score around 0.30. With Spring/Winter order volumes around 50% larger than Summer/Autumn, seasonal patterns also show a significant impact, as evidenced by the model's high weights for seasonal features.

It is closely followed by "delivery_distance" indicating that after seasonality, distance between customer and seller is the next biggest factor for orders being delivered late; confirming how geographical friction has a compounding effect, increasing exposure to handling points and transit uncertainty tenfold with the increase in distance. For this reason, proximity optimization is equally important as seasonal planning.



Moving forward, we would recommend the following implementations. First, increasing the workforce and optimizing the routes before spring/winter peaks. Second, strategically placing warehouses in high order volume locations to shorten the distance between sellers and customers.